Guess Market

Exercise 3 - Client-Server Prediction Market Application

README and Submission Documentation

Bonuses Implemented

Bonus	Points	Where to find it
Bonus - Chat	5 (up to 100)	Chat tab, third tab in the main window

Submitted By

Name	Nave Kluska
ID	324168160
Email	navekluska@gmail.com

GitHub Repository

https://github.com/NaveKluska/GuessMarket

Bonus - Details and How to Activate

Bonus - Chat

Every logged-in user can chat with every other logged-in user, entirely through the server - anyone connected sees exactly the same message stream, each message timestamped and colored per sender.

How to activate: nothing to switch on. The Chat tab is always there, the third tab in the main window. Type a message and press Send (or Enter) to post it; every client polls the server and shows new messages within about a second.

Running the Application

- Start the server first: copy `guessmarket.war` into Tomcat's `webapps` folder and start Tomcat (`bin\startup.bat`). It deploys automatically to `http://localhost:8080/guessmarket` - I assumed, as the exercise text allows, that the server's domain is `localhost:8080` and that the WAR's filename is not renamed after submission.
- Then run the client: unzip the client folder to any location (a path containing spaces is fine) and run `run.bat`.
- The login screen asks only for a name - there is no password and no separate sign-up step. Enter any name that is not already taken and I am taken straight to the Events screen. If the name is already in use, an inline message says so and lets me try again with a different one.
- Every client instance is hard-wired to `http://localhost:8080/guessmarket`; the server must already be running before a client is started.

System Overview

This project turns Guess Market into a client-server application. The engine now runs once, centrally, inside a Tomcat server, and every JavaFX client talks to it exclusively over HTTP - no client ever holds its own copy of the engine's state, and no client ever talks to another client directly; every exchange goes through the server.

Module structure

- `guess-market-dto` - unchanged in shape from Exercise 2: the plain, immutable DTOs that cross the engine/UI boundary, now also carried over the wire as JSON instead of being passed by reference in-process.
- `guess-market-engine` - the same UI-agnostic engine from Exercises 1-2, extended for this exercise with: chat message storage, the ability to accumulate events from more than one uploaded file instead of replacing them each time, and the new Exercise 3 XML schema and parser.
- `guess-market-server` - new for this exercise: a thin Tomcat/servlet layer that exposes the engine's capabilities as HTTP endpoints, and translates every result (or thrown exception) to and from JSON.
- `guess-market-client` - new for this exercise: the JavaFX client. It reuses Exercise 2's screens and layout wholesale, with the real additions being the login screen, replacing every direct engine call with an HTTP call to the server, live polling to keep every screen current, and the new Chat tab.

Server API Reference

Every endpoint below is served from the same base URL, `http://localhost:8080/guessmarket`, and everyone was re-confirmed directly against the running server while writing this table.

Method	Endpoint	Request Body	Notes
POST	`/users/register`	`{"name": "alice"}`	Register a user.
POST	`/users/deposit`	`{"userName": "alice", "amount": 100}`	Add cash to a balance.
GET	`/users`	-	List all users.
POST	`/upload?uploaderName=X` (change 'X' for actual name)	(raw XML file as the request body)	Upload events from an XML file.
GET	`/events`	-	List all events.
GET	`/events?name=X` (change 'X' for actual name)	-	One event's full details.
POST	`/events/open`	`{"mmName": "alice", "eventName": "X"}`	MM opens the event.
POST	`/events/close`	`{"mmName": "alice", "eventName": "X", "winningOptionIndex": 0}`	MM closes it, picks the winner.
POST	`/events/buy`	`{"memberName": "alice", "eventName": "X", "optionIndex": 0, "quantity": 5}`	Buy shares (LMSR events).
POST	`/events/order`	`{"userName": "alice", "eventName": "X", "optionIndex": 0, "side": "BUY", "price": 0.45, "quantity": 4}`	Bid/ask (Order Book events).
POST	`/events/create-lmsr`	`{"creatorName": "alice", "name": "X", "description": "...", "commission": 5, "commissionType": "ON_PURCHASE", "optionNames": ["Yes", "No"], "b": 100}`	Create one LMSR event directly.
POST	`/events/create-orderbook`	`{"creatorName": "alice", "name": "X", "description": "...", "commission": 5, "commissionType": "ON_PURCHASE", "optionNames": ["Yes", "No"], "allowMint": true, "initial": 50, "d": 1}`	Create one Order Book event directly.
POST	`/chat`	`{"userName": "alice", "message": "hi"}`	Post a chat message.
GET	`/chat`	-	List chat messages.

Every response is JSON. A failed request returns `{"error": "..."}` *with HTTP 400 for a validation error or HTTP 500 for an unexpected one.* `commissionType` *is* `ON_PURCHASE` *or* `ON_CLOSE`*;* `side` *is* `BUY` *or* `SELL`*.*

Design Choices and Assumptions

Where the specification left a decision open, the choices below were made deliberately, and are explained here as the specification requests.

File upload commits all-or-nothing

If any event inside an uploaded file has a name that already exists anywhere in the system - from an earlier upload by anyone, not just this uploader - the whole file is rejected with a message naming the exact duplicate. None of that file's events are added, not even the ones that were otherwise fine, since a partially-applied upload seemed more likely to confuse a user than help them.

Event names and usernames are both compared case-insensitively

The Exercise 3 schema dropped the event id, so an event is now identified by its name alone. Following the specification's general rule that all English input is case-insensitive ("MoMo" = "mOmO") - the same rule Exercise 2 already applied to usernames and option names - two event names that differ only in letter case count as the same event: while an event named "Rain" exists, uploading or creating "rain" is rejected, whether within one file or across separate uploads, and the error names the existing event. Usernames follow the same rule, so "Bob" and "bob" are the same person.

Polling re-fetches everything and diffs on the client

Rather than delta-fetching, at a 1-second interval - within the specification's stated up-to-two-seconds allowance. Each client asks the server for the full current picture (every event, every user, every chat message) on every tick, and only touches whatever part of the screen actually changed, instead of asking the server for "what changed since my last request."

A disabled control explains why, rather than just disabling

Buy/Sell/Deposit only work while viewing your own page, and Open/Close Event only works for an event's actual Market Maker; everywhere else the control stays visible but inert, with a tooltip naming whose action it actually is - so a user browsing someone else's page understands why a button does nothing, instead of assuming it is broken.

Order Book minting has no same-user restriction

Minting matches purely on price and quantity, exactly as Appendix B describes it; nothing in that description requires the two matched orders to belong to different people. I checked this deliberately, since a single user submitting opposing Buy orders on two different options can mint against themselves, and decided to leave the engine's literal, specification-following behavior as-is rather than add a same-user exclusion the exercise never asked for.

Other users' info: the required minimum is always visible, the rest is read-only

The specification draws a hard line here: a user's own account screen is private, but for every other user in the system it must additionally show their username, their account balance, and whether they are a Market Maker – nothing says it has to stop there, and nothing says how it has to be revealed. The left-hand user list satisfies that literally, for every row, with no click required: a name, a balance pill, and a MARKET MAKER pill when it applies. Selecting a user goes further and opens the same detailed view Exercise 2 used for your own account – which events they hold shares in, and how many – reusing exactly the components the specification asks projects to reuse, but with every Buy/Sell/Deposit/Open/Close control disabled, since a user may still only ever act on their own behalf. The extra detail is read-only visibility layered on top of the required minimum, never additional capability.

Out-of-range Spinner input is clamped, not rejected

Every bounded numeric field in the app – commission %, LMSR's b, Order Book's initial and d, quantities, price, deposit amount – is a plain JavaFX Spinner with a min/max range. Its arrows can never leave that range, but typing a value directly and confirming it (a negative commission, for instance) isn't rejected outright: JavaFX's own Spinner clamps it to the nearest legal bound instead – 0, in that example – the same built-in behavior the platform control already has everywhere it's used in this app. This wasn't worth overriding with extra validation on top of it, since resolving an out-of-range number to the closest legal one is itself a reasonable, unsurprising outcome.

Chat colors are assigned per-client, not stored on the server

Each username is deterministically hashed into a color locally, inside each client, rather than the server assigning and broadcasting one. Java's `String.hashCode()` is fixed by the language specification, so the same username always produces the same color on every machine regardless - this kept the server simpler and needed no new endpoint, without actually losing consistency between clients.

New Classes Documentation

A general, non-exhaustive overview of the main classes added or substantially changed for this exercise, grouped by module.

Server (new module)

- `ServerState` - holds the single `MarketEngine` instance shared for the server's entire lifetime. Every servlet reads through this to reach the one engine every client ultimately talks to.
- `JsonServlet` - the abstract base every servlet extends. Centralizes translating a thrown exception, or a normal result, into the same JSON success/error envelope, so no individual servlet repeats its own try/catch/write-error boilerplate.
- `JsonResponses`, `Requests` - small static helpers, shared by every servlet, for writing a JSON success or error body and for reading a JSON request body respectively.
- `EventsServlet`, `UsersServlet`, `UploadServlet`, `ChatServlet` - one servlet per resource, each translating its HTTP verbs into the matching `MarketEngine` calls: listing and detailing events, trading, opening and closing, depositing cash, uploading a file, and posting or listing chat messages.

Client (new module)

- `ApiClient` - the client's only channel to the server: one static method per server capability, each issuing an HTTP request (via OkHttp) and parsing the JSON response (via Gson) back into the exact same DTOs Exercise 2's engine used to hand the UI directly.
- `Session` - holds the one fact a client needs about itself once logged in: its own username.
- `ApiException` - thrown by `ApiClient` whenever the server's response is an error envelope. Its message is exactly the text the engine's own exception carried on the server side, so the UI can catch and display it exactly as Exercise 2's `MainController` used to catch the engine's own exceptions directly.
- `OrderSide`, `CommissionType` - small client-local enums mirroring the engine's own, needed only so the UI can build a request payload without the client module depending on the engine module at all.
- `LoginController` - the new screen behind `login.fxml`: takes a name, registers it against the server, and opens the main window on success.
- `MainController` - Exercise 2's own controller, ported close to wholesale, with every direct engine call replaced by the matching `ApiClient`/HTTP call, plus the login-driven `Session`, the polling loop that keeps every screen live, and the new Chat tab.